Couture Atelier Intelligence

Complete ELI5 Study Guide and Interview Preparation Manual

Project type: Luxury retail analytics portfolio simulation
Fictional brand: Couture Atelier
Target roles: Fashion data analyst, retail data scientist, customer analytics scientist, inventory analytics analyst, merchandising analytics analyst
Last updated: September 2026

How to Use This Guide

This guide explains the entire project from the beginning, in simple language first and technical language second.

Study it in this order:

1. Learn the one-minute explanation.
2. Learn the business problem.
3. Learn the three data layers.
4. Learn the pipeline from raw data to dashboard.
5. Study each model and metric.
6. Practice the interview questions.
7. Review the limitations and future improvements.

You should be able to explain the project at three levels:

- **30 seconds:** what the project does.
- **3 minutes:** how the pipeline and models work.
- **15 minutes:** technical decisions, validation, limitations, and business impact.

Part 1: The One-Minute Explanation

The simple version

Couture Atelier Intelligence is a decision-support system for a fictional luxury fashion retailer.

It helps answer four questions:

1. **What will customers buy?**
   Demand forecasting.
2. **Where should products be placed?**
   Inventory allocation, transfers, and replenishment.
3. **Which customers may need attention?**
   Customer lifetime value, churn prediction, and behavioral segmentation.
4. **What should a sales associate do next?**
   Product recommendations and consent-aware clienteling.

The dashboard connects those decisions in one executive interface.

The interview version

> I built Couture Atelier Intelligence, a reproducible luxury retail analytics platform using public retail observations and a clearly labeled synthetic luxury layer. The system transforms transaction data into customer, product, and demand marts; forecasts product-country demand; estimates customer value; predicts churn; creates behavioral segments; recommends inventory transfers and replenishment; and produces consent-aware clienteling actions. I used chronological validation, explicit provenance, fail-closed data checks, and a no-action option so the system supports responsible luxury retail decisions rather than blindly maximizing outreach or revenue.

The most important sentence

> The project is not pretending to have Louis Vuitton's private data. Public data supports the analytical methodology, while boutique, luxury product, inventory, consent, and clienteling fields are explicitly synthetic.

That sentence demonstrates judgment and protects the credibility of the project.

Part 2: Why This Project Exists

The business problem

Luxury retail is not only about selling more units.

A luxury house must balance:

- Product desirability
- Scarcity and exclusivity
- Availability in the right location
- Full-price selling
- Customer relationship quality
- Personalization without intrusion
- Inventory efficiency
- Service quality
- Brand consistency

A normal retail system might ask:

> How can we maximize conversions?

A luxury retail system should ask:

> How can we make the right product available to the right client in the right place, while protecting the experience and the brand?

Why generic machine learning is not enough

A model can have excellent accuracy and still be commercially wrong.

For example:

- It may recommend a product that is out of stock.
- It may contact someone who did not consent.
- It may treat a return as a purchase.
- It may interpret a stockout as lack of demand.
- It may over-contact a high-value client.
- It may optimize sales while damaging exclusivity.

This project therefore treats business rules, data lineage, consent, and explainability as part of the product.

Part 3: What the Project Is and Is Not

What it is

- A portfolio simulation.
- A reproducible analytics pipeline.
- A demonstration of retail, customer, forecasting, and decision-support skills.
- A bridge between public data science and luxury retail business logic.
- A system designed to be discussed with merchandising, retail, clienteling, and technology teams.

What it is not

- It is not Louis Vuitton data.
- It is not LVMH internal data.
- It is not a claim about private luxury-company performance.
- It is not a production CRM.
- It is not an automatic marketing system.
- It is not a causal experiment.
- It is not evidence that the simulated numbers represent real luxury demand.

Why the fictional name matters

Couture Atelier is an invented project identity. It avoids implying affiliation with a real luxury brand and makes it clear that the luxury operating layer is simulated.

Part 4: The Three Data Layers

The project has three important kinds of data.

Layer 1: Observed public data

This is data downloaded from public datasets.

Main source:

- UCI Online Retail II

Other planned or supporting sources:

- H&M Personalized Fashion Recommendations
- M5 Forecasting Accuracy
- Fashion Product Images Dataset
- Google Trends

The UCI dataset is the source currently acquired and used throughout the working pipeline.

Layer 2: Derived data

Derived data is calculated from observed data.

Examples:

- Net revenue
- Recency days
- Purchase frequency
- Rolling demand averages
- Churn probability
- Segment assignments
- Forecast error
- Transfer recommendations

Derived data is not directly observed. It is produced by code and can be regenerated.

Layer 3: Synthetic data

Synthetic data is generated by project rules.

Examples:

- Couture Atelier customer IDs
- Fictional product categories
- Fictional boutiques
- Luxury price tiers
- Inventory positions
- Consent status
- Warehouse replenishment
- Transfer decisions

Synthetic data allows the project to demonstrate luxury-specific use cases without exposing or inventing private company data.

Provenance labels

Every important output uses a provenance class:

Label	Meaning
`observed_public`	Loaded from a public source
`derived`	Calculated from source data
`synthetic`	Generated by simulation rules
`scenario`	A user-controlled assumption or what-if setting

When asked whether a result is real, answer using this vocabulary.

Part 5: The Source Dataset

UCI Online Retail II

The UCI dataset contains approximately 1.07 million transaction rows from a UK-based online retailer between December 2009 and December 2011.

Important fields:

- `Invoice`: invoice identifier
- `StockCode`: product identifier
- `Description`: product description
- `Quantity`: number of units
- `InvoiceDate`: transaction timestamp
- `Price`: unit price
- `Customer ID`: customer identifier
- `Country`: customer country

Why it was useful

It provides enough information to demonstrate:

- Customer segmentation
- Revenue analysis
- Repeat-purchase behavior
- Product demand
- Churn labeling
- CLV estimation
- Country-level demand analysis

Why it is not a perfect luxury dataset

The source is online giftware, not luxury fashion.

It does not contain:

- Boutique IDs
- Luxury product categories
- Store inventory
- Appointments
- Clienteling actions
- Product images linked to transactions
- Luxury price tiers
- Full customer consent history

That is why the project uses it as a methodological foundation and adds a separate synthetic luxury layer.

Source cleaning decisions

Exact duplicates

Exact duplicate rows were removed before staging.

Why?

If the same row appears twice because of a source duplication issue, counting both rows would artificially inflate revenue and demand.

The pipeline records how many duplicate rows were found and removed.

Cancellations

Invoices beginning with `C` are treated as cancellation or return events.

They remain in the staged data for auditability, but they are not treated as normal purchases.

Adjustments

Some rows represent accounting or stock adjustments, such as:

- Damages
- Lost items
- Bad debt
- Zero-price negative quantities
- Adjustment invoice codes beginning with `A`
- Product code `B`

These rows are retained and labeled as `is_adjustment`, but excluded from commercial sales metrics.

Missing customer IDs

Rows with missing customer IDs remain useful for:

- Product demand
- Country demand
- Inventory analysis

But they are excluded from customer-level features because there is no reliable person to attach the purchase to.

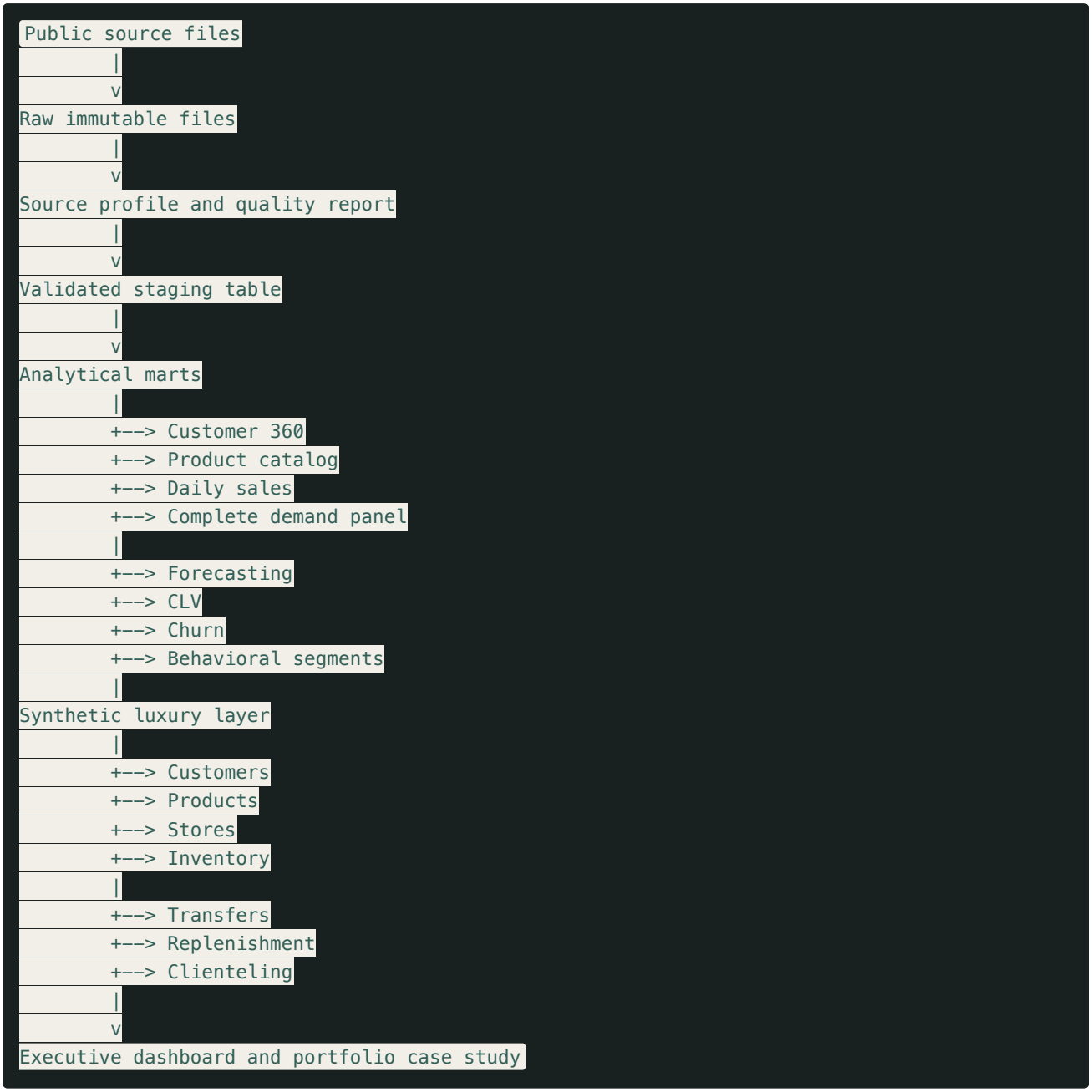
Invalid records

The staging layer fails if it finds:

- Missing required keys
- Invalid dates
- Invalid quantities
- Negative prices that are not adjustments
- Negative quantities that are not cancellations or recognized adjustments

This is called fail-closed validation: the pipeline stops rather than quietly producing questionable results.

Part 6: The Full Pipeline



Repository map

```
Couture Atelier Intelligence/
├── data/
│   ├── raw/
│   ├── staging/
│   ├── synthetic/
│   └── processed/
├── scripts/
├── tests/
├── docs/
├── dashboard/
├── configs/
├── src/
├── notebooks/
├── sql/
├── README.md
└── pyproject.toml
```

Main scripts

Script	Job
`ingest_uci_online_retail.py`	Downloads and registers UCI data
`profile_uci_online_retail.py`	Profiles sheets, dates, missing values, cancellations
`build_uci_staging.py`	Cleans, validates, and normalizes transactions
`build_analytical_marts.py`	Creates customer, product, and daily-sales marts
`build_demand_panel.py`	Adds explicit zero-demand dates
`build_baselines.py`	Creates RFM/CLV baseline and seasonal forecast benchmark
`build_demand_model.py`	Trains the stronger demand model
`build_synthetic_luxury_layer.py`	Creates fictional luxury customers, products, stores, inventory
`optimize_inventory.py`	Creates transfers and replenishment recommendations
`build_client_clv.py`	Creates the improved as-of CLV snapshot
`build_churn_and_segments.py`	Builds churn predictions and behavioral segments
`build_clienteling.py`	Creates consent-aware clienteling decisions
`dashboard/app.py`	Runs the executive Streamlit dashboard

Part 7: Staging and Analytical Marts

The staging table

The normalized staging table is:

`data/staging/uci_online_retail_ii.parquet`

Important normalized columns:

- `invoice_id`
- `product_id`
- `product_description`
- `quantity`
- `transaction_at`
- `unit_price`
- `line_revenue`
- `customer_id`
- `country`
- `is_cancelled`
- `is_adjustment`
- `has_customer_id`
- `data_provenance`

Line revenue

The basic formula is:

```
line_revenue = quantity × unit_price
```

This is a line-level amount, not necessarily a complete customer order value until invoice lines are aggregated.

Commercial metrics

The pipeline separates:

- Gross units
- Returned units
- Gross revenue
- Return value
- Net units
- Net revenue

The basic net formulas are:

```
net_units = gross_units - returned_units
net_revenue = gross_revenue - return_value
```

Customer 360 mart

Output:

`data/processed/customer_360.parquet`

Contains customer-level features such as:

- First purchase date
- Last purchase date
- Order count
- Line count
- Gross units
- Returned units
- Gross revenue
- Return value
- Net units
- Net revenue
- Distinct products
- Primary country

Purchase dates are calculated from actual purchases, not cancellations.

Product catalog mart

Output:

`data/processed/product_catalog.parquet`

Contains product-level summaries:

- First seen date
- Last seen date
- Transaction-line count
- Unique customers
- Gross units
- Returned units
- Gross revenue
- Return value
- Net units
- Net revenue

Daily sales mart

Output:

`data/processed/daily_sales.parquet`

Grain:

```
one row = one date × one product × one country
```

This is useful for country-level product demand, but it is sparse: rows exist only when activity was observed.

Part 8: The Complete Demand Panel

Why the sparse daily table was a problem

Suppose a product did not sell on Tuesday.

There are two possible meanings:

1. It was available but nobody bought it.
2. It was not represented in the source table because there was no transaction.

A forecasting model needs an explicit row for the date so it can learn from zero demand.

What the demand panel does

The demand panel creates a row for every date between the first and last active date of each product-country pair.

Output:

`data/processed/demand_panel.parquet`

Important flag:

- `is_observed = true`: original source row existed.
- `is_observed = false`: a zero-demand row was filled in by the pipeline.

Current size:

- 28,804 product-country keys
- 5,773,244 date rows
- 5,177,176 filled zero-demand rows

Why this matters for luxury retail

A luxury product may sell intermittently. The model must distinguish:

- Popular product
- Rare product
- New product
- Product with no recent demand
- Product unavailable due to stockout

The demand panel solves only the missing-date problem. It does not yet solve stockout-censored demand; that is why the synthetic inventory layer matters.

Part 9: Demand Forecasting

The business question

> How many units of each product should we expect to sell in each product-country series over the next period?

The current public-data model forecasts product-country demand, not yet true boutique-level luxury demand.

Baseline model: seasonal naive

The baseline uses demand from seven days earlier as the forecast.

Example:

```
Forecast for Tuesday = actual demand from last Tuesday
```

Why use a simple baseline?

Because a complicated model is only useful if it beats a simple alternative.

Baseline results on the 28-day holdout:

- MAE: 6.88 units
- RMSE: 242.30 units
- Bias: 5.24 units

Stronger model

The model is:

`HistGradientBoostingRegressor`

It is trained on:

```
log1p(gross_units)
```

Why log-transform the target?

Retail demand is usually uneven:

- Many rows have zero or small demand.
- A few products have very large demand spikes.

The logarithm compresses extreme values so the model is not dominated by the largest transactions.

After prediction, the inverse transformation is applied:

```
predicted_units = expm1(model_prediction)
```

Predictions are clipped to be nonnegative.

Model features

Lag features

- `lag_1`: demand yesterday
- `lag_7`: demand seven days ago
- `lag_14`: demand fourteen days ago
- `lag_28`: demand twenty-eight days ago

A lag is simply an old value used to help predict a new value.

Rolling features

- Seven-day rolling mean
- Twenty-eight-day rolling mean

Important: rolling features use prior observations only. The current day's target cannot leak into its own feature.

Calendar features

- Day of week
- Month
- Day of month

These help capture weekly and seasonal patterns.

Lifecycle features

- Product-country active age
- Days since previous sale

These help distinguish a new product from a mature product and an active product from a dormant one.

Leakage prevention

The model uses a chronological split:

```
training dates < cutoff date
testing dates >= cutoff date
```

The last 28 calendar days are held out.

The test data is never used to train the model.

The code also rejects:

- Invalid dates
- Duplicate product-country-date keys
- Missing required columns

Stronger model result

- MAE: 5.79 units
- RMSE: 241.93 units
- Bias: 4.33 units

Compared with the seasonal baseline, MAE improved from 6.88 to 5.79, approximately a 15.7% reduction.

How to explain the result correctly

Say:

> The feature-based model improves the public-data forecasting benchmark on a chronological holdout.

Do not say:

> The model will reduce Louis Vuitton stockouts by 15.7%.

The first statement is supported. The second is not.

Forecasting limitations

The model does not yet include:

- Luxury product attributes
- Price elasticity
- Promotions
- Boutique inventory availability
- Store-level demand
- External fashion trends
- Prediction intervals
- Intermittent-demand-specific models
- Hierarchical reconciliation

These are future improvements, not hidden capabilities.

Part 10: Synthetic Luxury Layer

Why synthetic data was needed

The public transaction data does not have the fields needed for luxury retail decisions.

The synthetic layer creates a fictional operating environment with:

- Clients
- Products
- Boutiques
- Product attributes
- Consent
- Inventory

Synthetic customers

Output:

`data/synthetic/customers.parquet`

Fields include:

- `customer_id`
- `home_region`
- `signup_date`
- `acquisition_channel`
- `consent_status`
- `data_provenance`

Customer IDs are remapped to forms such as:

```
CA-CLIENT-12345
```

No names, emails, addresses, ethnicity, or inferred wealth are used.

Synthetic products

Output:

`data/synthetic/products.parquet`

The project selects 100 hero products from the source product universe by derived net revenue, then maps them to fictional luxury products.

Fields include:

- Category
- Subcategory
- Material
- Color family
- Collection
- Price tier
- Launch date
- List price
- Heritage flag

Important nuance:

The source product IDs help select the universe, but the luxury product attributes are synthetic. They are not factual descriptions of real products.

Synthetic stores

Output:

`data/synthetic/stores.parquet`

Six fictional locations:

- Paris flagship
- London boutique
- Milan boutique
- New York flagship
- Tokyo boutique
- Dubai airport location

Store attributes include:

- Region
- Store type
- Floor-area band
- Tourism index
- Affluence index

These indices are synthetic scenario inputs, not claims about actual cities.

Synthetic inventory

Output:

`data/synthetic/inventory_daily.parquet`

The simulation covers:

- 90 days
- 6 stores
- 100 products
- 54,000 inventory rows

The inventory equation is:

```
closing_stock = opening_stock
              + units_received
              + transfer_in
              - units_sold
              - transfer_out
```

Every generated row is checked against this equation.

Demand simulation

The current synthetic demand uses a gamma-Poisson process.

ELI5 explanation:

- The gamma distribution creates different demand intensities.
- The Poisson distribution turns that intensity into unit counts.
- Some store-product pairs sell more often than others.
- Inventory limits the number of units that can be sold.

The current synthetic layer is independent of observed public product-country demand. This is documented and is an important limitation.

Part 11: Inventory, Transfers, and Replenishment

The business question

> Given current stock and recent demand, where should inventory move and how much should the warehouse replenish?

Target stock

For each store-product pair:

```
target_stock = ceil(daily_demand_rate × horizon_days + safety_units)
```

Current settings:

- Lookback: 14 days
- Horizon: 14 days
- Safety stock: 2 units
- Minimum display stock: 1 unit

The minimum display stock reflects a luxury-specific concern: a store should not look empty simply because inventory was moved out.

Donors and recipients

A donor has stock above its target.

A recipient has stock below its target.

Transfers are matched by:

- Largest donor surplus first
- Largest recipient deficit first
- Same product only
- No store-to-itself transfer

Constraints

Current constraints:

- Donor cannot transfer more than its surplus.
- Donor is capped at 12 transfer units.
- Every target retains at least one display unit.
- Warehouse replenishment is capped at 200 units.

Replenishment

After constrained transfers are allocated, remaining deficits may be covered by the central warehouse.

This is not full procurement optimization yet. It is a transparent warehouse-capacity scenario.

Current results

- 13 transfer recommendations
- 59 transfer units
- 14 replenishment recommendations
- 200 replenishment units
- Baseline projected stockout rate: 69.85%
- Recommended projected stockout rate: 69.36%
- Improvement: 0.49 percentage points

Why the improvement is modest

The synthetic system has:

- High stockout pressure
- Limited transfer capacity
- Limited warehouse capacity
- No transfer cost
- No lead time
- No service-level optimization

A modest improvement is more credible than claiming an unrealistic result.

What remains for a more advanced optimizer

- Transfer cost
- Transfer lead time
- Warehouse lead time
- Minimum order quantities
- Reorder points
- Service-level objectives
- Full-price availability
- Product priority weights
- Regional restrictions
- Sensitivity analysis

Part 12: Client Intelligence

The business question

> Which clients are valuable, which may leave, and how are their behaviors different?

The project has three client-intelligence components:

1. CLV
2. Churn prediction
3. Behavioral segmentation

12.1 Customer Lifetime Value

ELI5 explanation

CLV estimates how much value a customer may generate in the future.

It is not a crystal ball. It is a structured estimate based on past behavior.

As-of date

The model uses a cutoff date.

Only purchases on or before the cutoff are allowed to create features.

Future purchases are kept separate for evaluation.

This prevents a serious error called target leakage.

Features

- Order count
- Historical revenue
- First purchase date
- Last purchase date
- Recency days
- Active customer age
- Average order value
- Purchase rate
- Product breadth
- Return rate

Purchase rate

A simple purchase rate is approximately:

```
orders_per_day = order_count / active_days
```

Expected future orders are adjusted for:

- A small prior order count
- Recency decay

Recency factor

The current model uses exponential decay:

```
recency_factor = exp(-recency_days / 180)
```

This means a customer who has been inactive longer receives a lower expected future value.

Smoothed order value

The model uses a small revenue prior:

```
smoothed_order_value = (historical_revenue + prior_revenue)
                       / (order_count + 1)
```

This prevents very small histories from producing unstable values.

CLV formula

Conceptually:

```
predicted_clv_90d = expected_orders_90d
                  × smoothed_order_value
                  × (1 − return_rate)
```

Current result

- 5,283 customers evaluated
- 90-day horizon
- MAE: 467.26 revenue units
- Prediction/realization correlation: 0.686

The model underpredicts aggregate future revenue. This is honestly documented as a calibration target.

What to say in an interview

> I began with an interpretable CLV baseline rather than pretending a complex model was automatically better. It uses an as-of feature window and evaluates against a strictly future 90-day revenue window. The correlation is useful but the aggregate underprediction shows that calibration and probabilistic CLV are still needed.

12.2 Churn prediction

ELI5 explanation

Churn means the customer did not purchase again during a defined future period.

Current definition:

```
churned_90d = 1 if no eligible purchase occurs in the next 90 days
churned_90d = 0 if at least one eligible purchase occurs
```

Time-based setup

The model is trained using an earlier as-of date and evaluated using a later as-of date.

Current dates:

- Training as-of date: March 14, 2011
- Test as-of date: June 12, 2011

This is better than randomly splitting customers because random splitting can mix future and past behavior.

Model

The current churn model is logistic regression with:

- Standardized features
- Balanced class weights
- Deterministic random seed

Features

- Recency days
- Order count
- Historical revenue
- Distinct products
- Purchase cadence
- Average order value
- Lines per order
- Product breadth

Metrics

- ROC-AUC: 0.816
- Average precision: 0.889
- Brier score: 0.173

Metric meanings

ROC-AUC

Measures whether the model ranks a future churner above a future non-churner.

A value of 0.5 is roughly random ranking.

A value of 1.0 is perfect ranking.

Average precision

Useful when the positive class is important or imbalanced. It summarizes precision/recall behavior across thresholds.

Brier score

Measures probability quality. Lower is better.

A model can rank customers well but still have poorly calibrated probabilities. That is why Brier score matters.

Important limitation

The test churn rate is high because of the definition and time window. The model result should not be interpreted as a real luxury retention rate.

Future improvement:

- Probability calibration
- Multiple rolling windows
- Cost-sensitive threshold selection
- Retention experiment or uplift modeling

12.3 Behavioral segmentation

ELI5 explanation

Segmentation groups customers with similar behavior.

The goal is not to label people socially. The goal is to make repeated purchasing patterns easier to understand.

Method

The project uses K-means clustering.

K-means works like this:

1. Choose a number of groups.
2. Place customers into groups based on distance in feature space.
3. Recalculate each group center.
4. Repeat until assignments stabilize.

Current number of segments:

```
K = 5
```

Features

- Recency
- Order count
- Historical revenue
- Distinct products
- Purchase cadence
- Average order value
- Lines per order
- Product breadth

Revenue features are log-transformed before clustering so extremely large customers do not dominate all groups.

Features are standardized so different units are comparable.

Output

- 5,283 customer assignments
- 5 segment profiles
- Average feature values per segment
- Customer count per segment

How to interpret segments

The numeric cluster IDs are not meaningful by themselves.

You must inspect the profile table and name groups based on behavior, for example:

- High value, frequent, recent
- High value, lapsed
- Emerging, low history
- Broad-category shoppers
- Narrow-category specialists

Never claim that a segment represents wealth, ethnicity, personality, or social class.

Future improvement

- Segment stability over time
- Compare K values
- Use silhouette score
- Add category and channel behavior
- Give segments business names only after inspecting profiles

---

Part 13: Recommendations and Clienteling

The business question

> What should a sales associate consider next for a consenting client?

The clienteling system is decision support, not automatic outreach.

Inputs

- Consent status
- Current inventory availability
- Purchase history
- Predicted CLV
- Churn probability
- Product ranking

Candidate selection

The system:

1. Looks at the fictional product universe.
2. Removes products already purchased by the customer.
3. Checks whether the product is currently in stock.
4. Selects an eligible product.
5. Chooses an action based on value, churn, and consent.

Action rules

No consent

Action:

```
no_action
```

Reason:

```
personalization consent unavailable
```

This is a hard privacy rule.

High churn probability

Action:

```
re_engagement
```

High CLV

Action:

```
private_appointment
```

Other consenting customer

Action:

```
product_suggestion
```

No eligible product

Action:

```
no_action
```

Current output

- 5,942 client decisions
- 4,766 consented clients
- 1,176 no-action decisions
- 2,184 product suggestions
- 1,553 re-engagement actions
- 1,029 private-appointment actions

Why no-action matters

A luxury brand should not contact every possible client merely because a model found a possible product.

No-action is a valid recommendation when:

- Consent is absent.
- The product is unavailable.
- The customer was contacted recently.
- The recommendation is weak.
- The relationship should be left undisturbed.

What is missing

The current system does not yet have real or simulated outcomes such as:

- Message opened
- Appointment booked
- Product viewed
- Product purchased
- Client response
- Contact fatigue
- Associate workload

Those are important future improvements because a recommendation cannot be judged only by whether a product was selected.

Part 14: The Dashboard

How to run it

From the project root:

```
cd "/Users/nora/Desktop/jobs/Fashion Projects/Couture Atelier Intelligence"
"/Users/nora/Desktop/jobs/Fashion Projects/.venv/bin/streamlit" run dashboard/app.py --
server.headless true --server.port 8503
```

Then open:

```
http://localhost:8503
```

Port 8502 may already be occupied by another process. If so, use 8503 or another available port.

Dashboard design philosophy

The dashboard is intentionally restrained:

- Four views
- Few high-signal KPIs
- Clear section labels
- Explicit provenance
- Minimal decoration
- Luxury-inspired ivory, jade, gold, coral, and charcoal palette
- Serif display headings with practical sans-serif data text
- No automatic outreach
- No overcrowded KPI grid

View 1: Executive Pulse

Purpose:

Give an executive the full-system signal in seconds.

Shows:

- Client universe
- Projected stockout rate
- Forecast MAE
- Churn ROC-AUC
- Executive read
- Action mix

Important design decision:

Forecast MAE is shown with its percentage change against the seasonal baseline. This gives context instead of showing a number with no comparison.

View 2: Demand and Inventory

Purpose:

Show where inventory pressure exists and what should move.

Shows:

- Hero products
- Boutiques
- Transfer units
- Stockout rows
- Transfer ledger
- Stockout pressure by store
- Current operating constraints

The chart uses percentage formatting for stockout rates so the user does not need to mentally convert decimals.

View 3: Client Intelligence

Purpose:

Show customer value and risk without turning clients into opaque labels.

Shows:

- Predicted 90-day value
- High-risk clients
- Behavioral segment count
- CLV correlation
- Value/risk scatter plot
- Behavioral profiles

View 4: Clienteling

Purpose:

Give an associate or manager a ranked decision list.

Filters:

- All decisions
- Eligible actions
- Re-engagement
- Private appointment
- Product suggestion
- No action

Each row shows:

- Customer ID
- Action type
- Recommended product
- Predicted CLV
- Churn probability
- Reason

The dashboard explicitly says this is associate decision support, not automatic outreach.

Part 15: How to Explain the Plots

Action mix bar chart

Question it answers:

> How many clients fall into each next-best-action category?

Read it by comparing the bars, not by treating the chart as revenue impact.

It shows model decisions, not successful conversions.

Stockout pressure chart

Question it answers:

> Which stores have the greatest share of inventory rows flagged as stockouts?

It is a pressure indicator, not a complete lost-sales estimate.

A stockout rate can be influenced by:

- Demand simulation
- Starting stock
- Receipts
- Horizon
- Product mix

Value/risk map

X-axis:

- Predicted 90-day CLV

Y-axis:

- Churn probability

Interpretation:

- High value, high churn: possible priority for careful re-engagement.
- High value, low churn: protect the relationship and avoid unnecessary pressure.
- Low value, high churn: lower-cost or no-action strategies may be appropriate.
- Low value, low churn: maintain normal service.

The chart supports prioritization; it does not prove causal effects.

Behavioral profile chart

Question it answers:

> How do the average behaviors of the five segments differ?

Do not name a cluster solely from its number. Inspect its averages first.

Transfer ledger

Question it answers:

> What movement is recommended, from where, to where, and why?

It should be read together with:

- Donor capacity
- Recipient deficit
- Minimum display stock
- Warehouse replenishment
- Product importance

Part 16: Testing and Quality Assurance

Current test count

The project currently has **30 passing tests**.

Test categories

Staging tests

- Cancellation handling
- Missing customer IDs
- Duplicate removal
- Adjustment classification

Mart tests

- Return separation
- Purchase-date correctness
- Net values

Demand panel tests

- Zero-demand date creation
- Provenance

Forecasting tests

- Lag correctness
- Group isolation
- Rolling means
- Zero-demand recency
- Duplicate-key failure
- Missing-column failure
- Deterministic model behavior

Synthetic layer tests

- Anonymous IDs
- Consent field
- Product validity
- Inventory determinism
- Nonnegative stock
- Inventory reconciliation

Inventory tests

- Donor/recipient matching
- Stockout improvement
- Replenishment output
- Transfer capacity

Client intelligence tests

- Future-only CLV features
- CLV evaluation
- Churn label correctness
- Deterministic segmentation

Clienteling tests

- Consent gating
- In-stock recommendation
- Churn-aware action
- No-action behavior

Quality commands

Run tests:

```
"/Users/nora/Desktop/jobs/Fashion Projects/.venv/bin/python" -m pytest -q
```

Run lint:

```
"/Users/nora/Desktop/jobs/Fashion Projects/.venv/bin/ruff" check scripts tests dashboard
```

Compile key files:

```
"/Users/nora/Desktop/jobs/Fashion Projects/.venv/bin/python" -m py_compile dashboard/app.py
```

Why tests matter in an interview

The tests show that the project is not just a notebook with attractive charts.

They protect:

- Data assumptions
- Time boundaries
- Privacy rules
- Inventory equations
- Recommendation eligibility
- Reproducibility

Part 17: Metrics Cheat Sheet

Forecasting

MAE

Mean absolute error:

```
average(|actual − forecast|)
```

Easy to interpret in units.

RMSE

Root mean squared error:

```
sqrt(average((actual − forecast)^2))
```

Punishes large mistakes more heavily than MAE.

Bias

```
average(actual − forecast)
```

Positive bias here means the model tends to underpredict.

WAPE

Weighted absolute percentage error. Useful when total demand scale matters, but can be unstable when actual demand is near zero.

MASE

Mean absolute scaled error. Compares the model with a naive benchmark and is useful across series with different scales.

Classification

Precision

Of those predicted positive, how many were actually positive?

Recall

Of all actual positives, how many did the model find?

ROC-AUC

How well does the model rank positives above negatives across thresholds?

Average precision

Summarizes precision-recall behavior, especially useful for imbalanced outcomes.

Brier score

Measures the quality of probability predictions. Lower is better.

CLV

MAE revenue

Average absolute difference between predicted and realized future revenue.

Correlation

Measures whether higher predicted value tends to correspond to higher realized value.

Correlation is not calibration. A model can rank well while predicting the wrong absolute amounts.

Clustering

Segment count

How many groups were created.

Segment profile

Average feature values for each group.

Stability

Whether similar customers remain assigned to similar groups when the time window or random seed changes.

Part 18: Interview Questions and Answers

Q1. What problem does this project solve?

Answer:

It connects four luxury retail decisions: demand forecasting, inventory movement, customer intelligence, and clienteling. The goal is to help a fictional luxury retailer decide what demand may occur, where products should be available, which customers may need attention, and when no action is more appropriate.

Q2. Why did you use public data instead of claiming luxury-house data?

Answer:

Private luxury transaction and clienteling data is not publicly available and should not be fabricated. I used public data for the analytical foundation and clearly separated it from a synthetic luxury layer. That lets me demonstrate the methods while maintaining data integrity and privacy.

Q3. Why is the UCI dataset not enough by itself?

Answer:

It has transactions, products, customers, dates, prices, and countries, but it lacks boutiques, inventory, appointments, consent, luxury product attributes, and clienteling outcomes. It is strong for methodology but insufficient for the complete luxury retail use case.

Q4. How did you prevent leakage?

Answer:

I used chronological cutoffs. Features are created only from data available at the as-of date, while future windows are held out for evaluation. I also test that lag features do not cross product-country groups and that duplicate keys fail fast.

Q5. Why did you create a complete demand panel?

Answer:

The sparse transaction table only contains dates with observed activity. Forecasting needs explicit zero-demand dates. I generated each product-country pair's active date range and added zero rows with an `is_observed` flag, so missing dates are distinguishable from original observations.

Q6. Why did you choose gradient boosting?

Answer:

It handles nonlinear relationships between lag, calendar, lifecycle, and demand features while remaining practical for tabular retail data. I introduced it only after establishing a seasonal-naive benchmark.

Q7. Why did you log-transform demand?

Answer:

Retail demand is skewed: many small or zero values coexist with a few large spikes. `log1p` reduces the influence of extreme values and makes the regression problem easier to model. Predictions are transformed back with `expm1` and clipped at zero.

Q8. Is the 15.7% improvement a business impact result?

Answer:

No. It is a model MAE improvement on a public-data chronological holdout. It does not prove a 15.7% stockout or revenue improvement in a real luxury business.

Q9. What is your strongest model limitation?

Answer:

The demand model does not yet observe true availability. A low sale count may mean low demand or a stockout. The synthetic inventory layer begins addressing that, but a more advanced system would model censored demand explicitly.

Q10. How is CLV calculated?

Answer:

The baseline estimates expected future orders using historical purchase rate, active age, recency decay, and a small prior. It multiplies expected orders by a smoothed order value and adjusts for return rate. It is transparent and evaluated against future 90-day revenue.

Q11. What does churn mean here?

Answer:

Churn means no eligible purchase during the 90 days after an as-of date. It is an operational definition for this project, not a universal truth about customer relationships.

Q12. Why use logistic regression for churn?

Answer:

It is interpretable, fast, and produces probabilities. For a first time-based benchmark, that is more useful than jumping immediately to a black-box model.

Q13. Why use K-means?

Answer:

K-means creates understandable behavioral groups from standardized numeric features. It is a baseline, not a claim that customer behavior naturally has exactly five permanent groups.

Q14. Why is consent a hard gate?

Answer:

Clienteling is relationship management, not unrestricted targeting. A recommendation should never become an outreach action when consent is unavailable.

Q15. Why include no-action?

Answer:

In luxury, restraint can protect the relationship. No-action prevents the system from treating every client as an opportunity for contact and gives associates permission not to intervene.

Q16. What does the inventory optimizer do?

Answer:

It estimates recent daily demand, calculates target stock for a future horizon, identifies store-level surpluses and deficits, creates constrained transfers, and sends remaining deficits to a finite warehouse replenishment plan.

Q17. What inventory constraints exist?

Answer:

Minimum display stock, donor surplus, a 12-unit transfer capacity per donor, no self-transfer, product matching, and a 200-unit warehouse capacity.

Q18. What would you improve next?

Answer:

I would calibrate churn and CLV probabilities, evaluate across multiple rolling windows, add transfer cost and lead time, then add clienteling response outcomes. I would prioritize those before adding more visual complexity.

Q19. How would you deploy this?

Answer:

The current dashboard is a Streamlit portfolio application. In production, I would separate ingestion, feature generation, model serving, recommendation approval, and dashboard presentation. I would add access controls, audit logs, monitoring, model versioning, and human approval before any client-facing action.

Q20. What would you monitor in production?

Answer:

I would monitor data freshness, schema drift, missing customer IDs, forecast error, forecast bias, stockout rate, transfer feasibility, churn calibration, recommendation coverage, consent violations, contact frequency, and segment drift.

Part 19: Questions You Should Ask the Interviewer

Strong candidates also ask good questions.

Examples:

1. How does the team define success for retail analytics: forecast accuracy, availability, full-price sell-through, client retention, or a combination?
2. How are store and e-commerce data joined today?
3. How does the business distinguish low demand from out-of-stock demand?
4. What level of explainability do merchandising and retail teams expect?
5. How are clienteling recommendations approved and measured?
6. What privacy and consent constraints govern customer analytics?
7. Does the team prefer centralized models or market-specific models?
8. How are model outputs incorporated into associate workflows?

Part 20: What Is Actually Finished?

Completed

- Public source ingestion
- Source profiling
- Staging validation
- Duplicate handling
- Cancellation handling
- Adjustment handling
- Customer 360 mart
- Product catalog mart
- Daily sales mart
- Complete demand panel
- Seasonal forecast baseline
- Feature-based demand model
- Synthetic customers
- Synthetic products
- Synthetic stores
- Synthetic inventory
- Stockout simulation
- Constrained transfers
- Warehouse replenishment
- Improved CLV baseline
- Churn prediction
- Behavioral segmentation
- Product recommendation baseline
- Consent rules
- Next-best-action logic
- No-action logic
- Executive Streamlit dashboard
- Architecture documentation
- Executive presentation outline
- Interview case study
- 30 automated tests

Still worth improving

These are meaningful, but not required to explain the current project:

1. Churn probability calibration.
2. CLV calibration.
3. Multiple rolling-window evaluations.
4. Transfer cost and lead time.
5. Full-price availability objective.
6. Clienteling response and conversion simulation.
7. Category affinity.
8. Recommendation diversity.
9. Segment stability.
10. Visual similarity.

Do not claim these are already implemented. Say they are the next improvements.

Part 21: The Best Study Routine

Day 1: Business understanding

Study:

- One-minute explanation
- Business problem
- What the project is and is not
- Three data layers

Practice explaining the project without mentioning algorithms.

Day 2: Data pipeline

Study:

- UCI fields
- Cleaning decisions
- Staging columns
- Analytical marts
- Demand panel

Practice answering:

> What happens to a cancelled invoice?

Day 3: Forecasting

Study:

- Seasonal baseline
- Lag features
- Rolling means
- Log transform
- Chronological split
- MAE, RMSE, bias

Practice explaining why the stronger model is better without overstating the result.

Day 4: Synthetic layer and inventory

Study:

- Why simulation is necessary
- Inventory equation
- Target stock
- Transfers
- Replenishment
- Constraints

Practice calculating target stock with a simple example.

Day 5: Client intelligence

Study:

- CLV formula
- As-of date
- Future evaluation
- Churn definition
- ROC-AUC and Brier score
- K-means segmentation

Day 6: Clienteling and dashboard

Study:

- Consent gate
- Product eligibility
- Action hierarchy
- No-action behavior
- Dashboard views
- Plot interpretation

Day 7: Interview simulation

Answer the 20 questions aloud.

For every answer, include:

1. Business reason
2. Technical method
3. Validation method
4. Limitation

That four-part answer structure will make you sound prepared without sounding memorized.

Final Mental Model

Remember the project as one chain:

```
Observe demand
    -> clean the data
    -> build trustworthy features
    -> forecast demand
    -> simulate luxury stores and inventory
    -> move or replenish stock
    -> understand client value and risk
    -> respect consent
    -> recommend a useful action or no action
    -> show the decisions clearly in a dashboard
```

The project's strongest quality is not one model score.

It is the connection between:

- Data quality
- Forecasting
- Inventory
- Customer intelligence
- Privacy
- Human judgment
- Luxury brand restraint

That is the story you should carry into every interview.